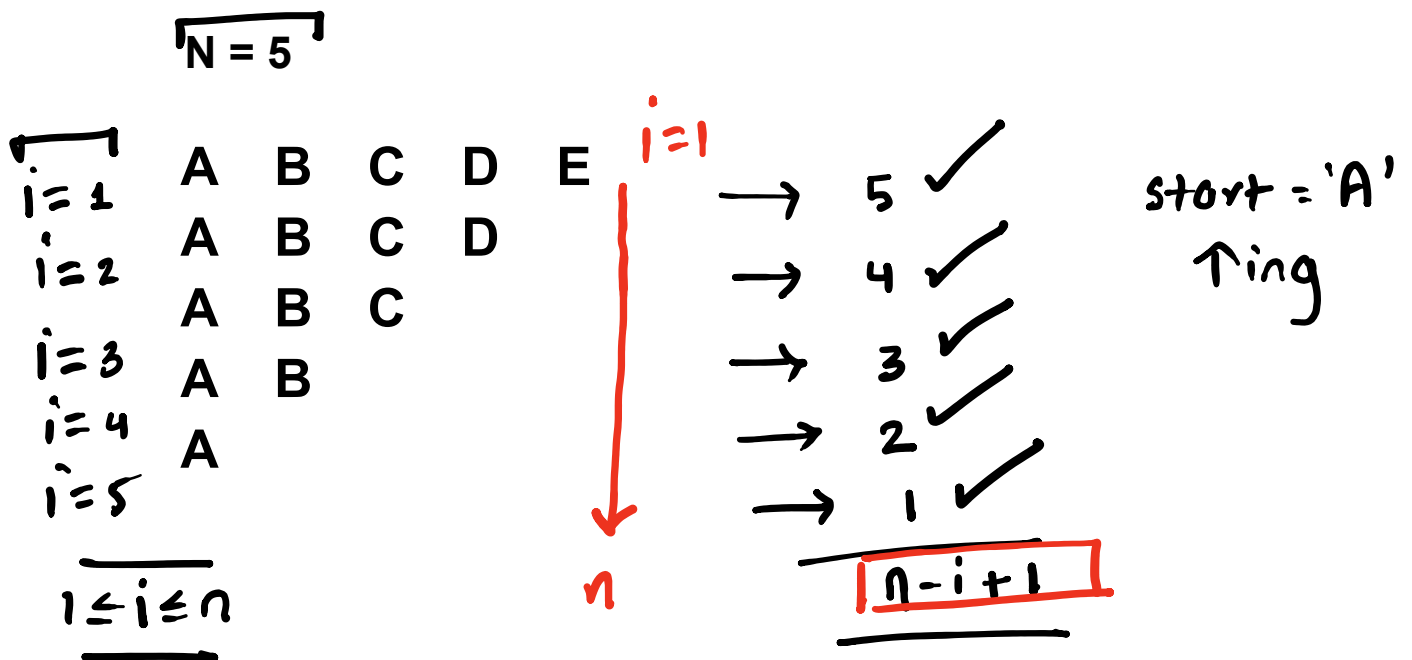


Pattern - 8

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example



Pattern - 9

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

```

      A B C D E E D C B A
     A B C D C B A
    A B C C B A
   A B B A
  A A
  
```

N = 5

```

1  A B C D E E D C B A
2  A B C D C B A
3  A B C C B A
4  A B B A
5  A A
  
```

$1 \leq i \leq n$

$i=1$

n

$n-i+1$

$n-i+1$

$\Rightarrow$ start = 'A' ↓ing
 ↑ing start =

[HW] Pattern - 10

Given a positive odd number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 9

P1

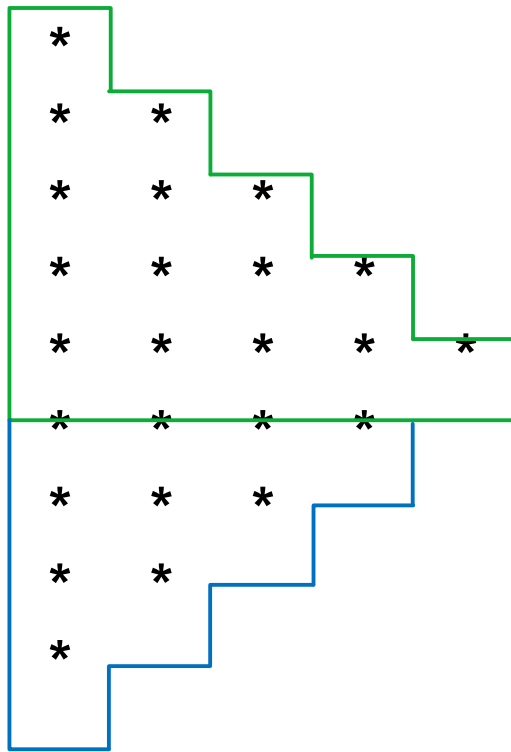
```

      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *
* * * * *
 * * * * *
  * * * *
   * * *
    * *
     *

```

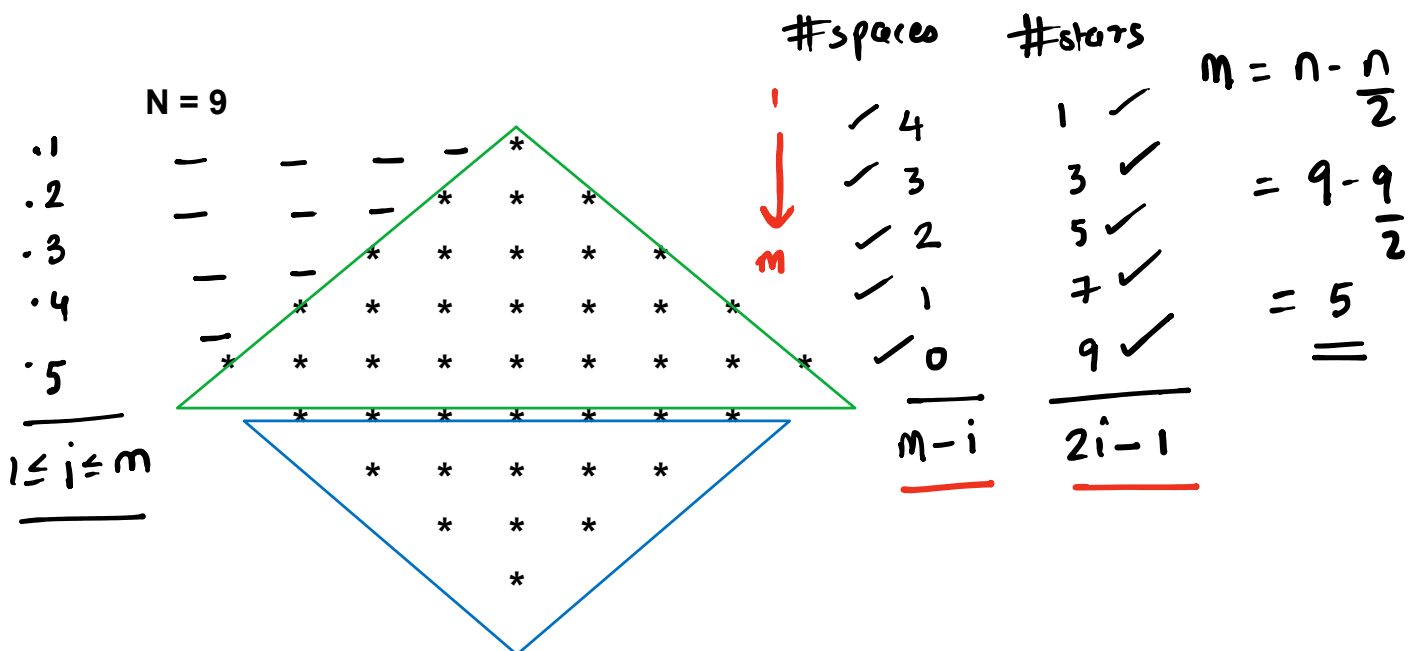
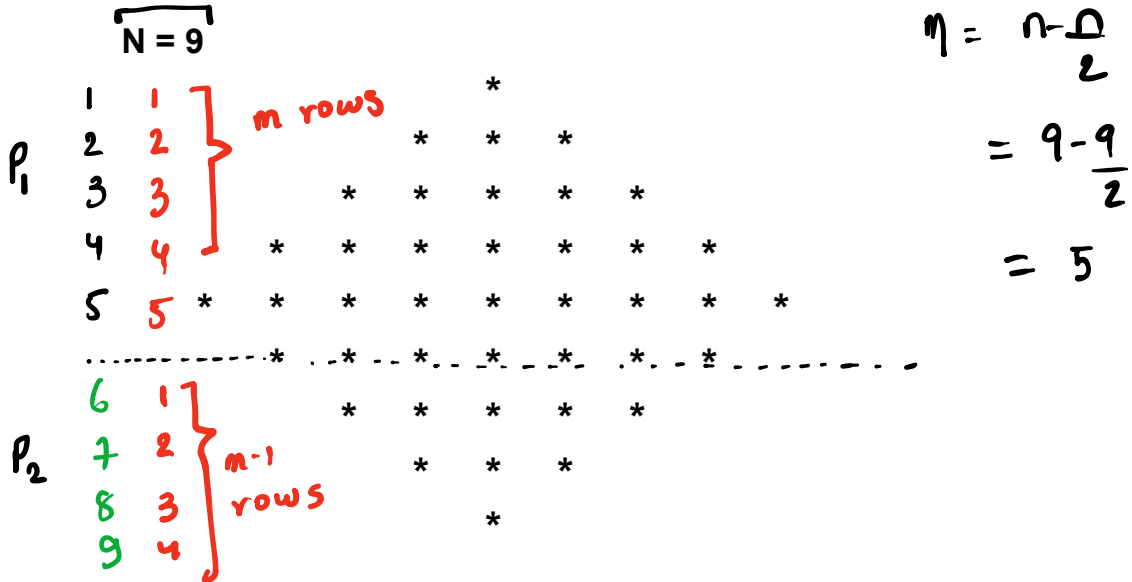
P2

N = 9



Given a positive odd number **N**, design an algorithm to **print** the pattern as shown in the example below.

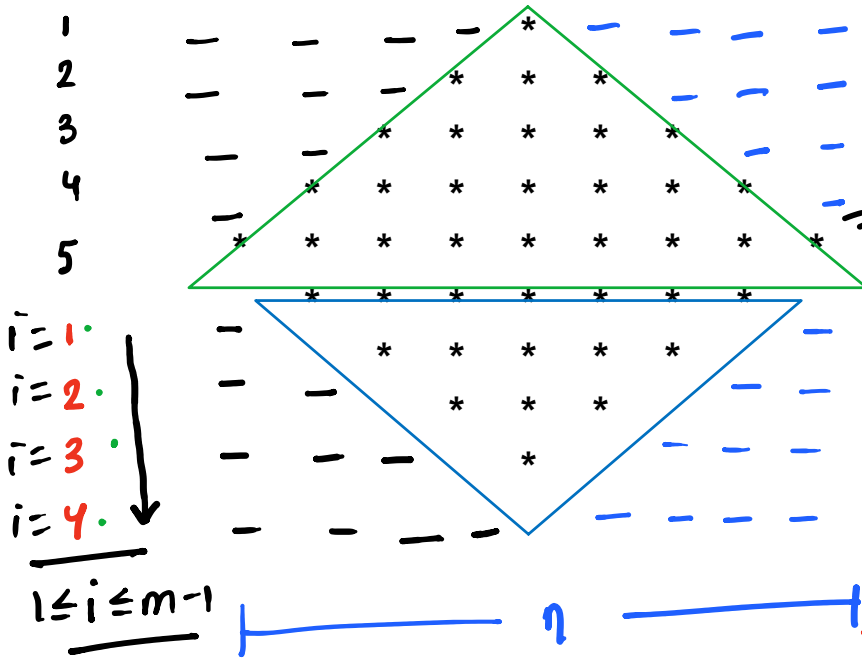
Example



$N = 9$

$$m = \frac{n - 1}{2} = \frac{9 - 1}{2} = 4$$

N = 9



$$m = n - \frac{n}{2} = 9 - \frac{9}{2} = 5$$

$$n = 9$$

$$m-i$$

$n-1$ rows

$$\left. \begin{array}{l} .4 \\ .3 \\ .2 \\ .1 \end{array} \right\}$$

[HW] Pattern - 12

Given a positive odd number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 9

```
* * * * * * * * *
* * * *   * * * *
* * *     * * * *
* *       * * * *
*         * * * *
* *       * * * *
* * *     * * * *
* * * *   * * * *
* * * * * * * * *
```

N = 9

P1

```
* * * * * * * * *
```

P2

```
* * * * _ * * * *
* * * _ _ _ * * *
* * _ _ _ _ * *
* _ _ _ _ _ _ *
* * _ _ _ _ _ * *
```

p3

*	*	*	—	—	—	*	*	*
*	*	*	*	—	*	*	*	*
*	*	*	*	*	*	*	*	*

p4

--	--	--	--	--	--	--	--	--

Bitwise Operators in C++

C++ offers six different **Bitwise** operators.

Bitwise Comparator	Example	Meaning
&	a & b	Bitwise AND
	a b	Bitwise OR
^	a ^ b	Bitwise XOR
~	~a	Bitwise NOT
<<	a << b	Bitwise Left Shift
>>	a >> b	Bitwise Right Shift

Bitwise AND (&) Operator

a = 25
b = 19

cout << (a & b);

25 = 16 + 8 + 1
↓ ↓ ↓
2⁴ 2³ 2⁰
= = =

19 = 16 + 2 + 1
↑ ↑ ↑
2⁴ 2¹ 2⁰

.	.	&
1	1	1
1	0	0
0	1	0
0	0	0

	4th	3rd	2nd	1st	0th
a = 25	1	1	0	0	1
b = 19	1	0	0	1	1
a & b	1	0	0	0	1

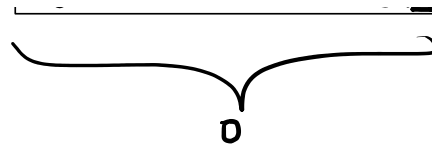
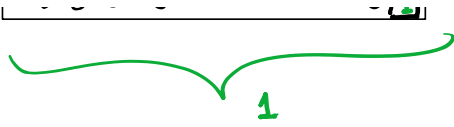
= 17

- $a \& 0 = 0$
- $a \& a = a$
- $a \& b = b \& a$ // commutative
- $a \& (b \& c) = (a \& b) \& c$ // associative
- $a \& 1 =$
 - 0th bit of a is set → 1 odd
 - 0th bit of a is not set → 0 even

$a \& 1$

use this to extract the 0th bit of any no.

$\Rightarrow a$	1	a	0
$\& \Rightarrow 1$	0000.....01	$\& 1$	0000.....01
res	0000.....01	res	0000.....00



2 → 10
 4 → 100
 6 → 110
 8 → 1000
 ⋮

3 → 11
 5 → 101
 7 → 111
 9 → 1001
 ⋮

Bitwise OR (|) Operator

$a = 25$
 $b = 19$

cout << (a | b);

1	1	1
1	0	1
0	1	1
0	0	0

⇒

	4th	3rd	2nd	1st	0th
a = 25	1	1	0	0	1
b = 19	1	0	0	1	1
a b	1	1	0	1	1

$$2^4 2^3 \quad 2^1 2^0 = 16 + 8 \quad 2 + 1 = 27$$

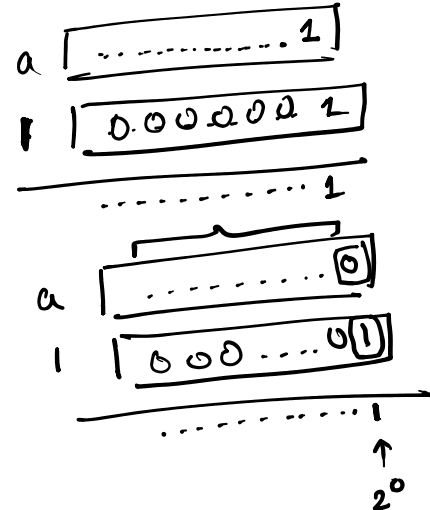
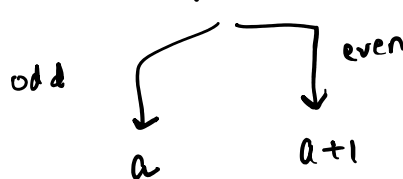
1. $a | b = b | a$

2. $a | (b | c) = (a | b) | c$

3. $a | a = a$

4. $a | 0 = a$

5. $a | 1$



Bitwise XOR (^) Operator

true ^ false = true

false ^ true = true

false ^ false = false

true ^ true = false

Bitwise XOR (^) Operator

$\text{false} \wedge \text{false} = \text{false}$
 $\text{true} \wedge \text{true} = \text{false}$

					^
→	1	1			0
	1	0			1
	0	1			1
→	0	0			0

⇒

	4th	3rd	2nd	1st	0th
a = 25	1	1	0	0	1
b = 19	1	0	0	1	1
a ^ b	0	1	0	1	0

(10)

- $a \wedge a = 0$
- $a \wedge 0 = a$
- $a \wedge b = b \wedge a$
- $a \wedge (b \wedge c) = (a \wedge b) \wedge c$

Bitwise NOT (~) Operator

	~
1	0
0	1

⇒

	4th	3rd	2nd	1st	0th
a = 25	1	1	0	0	1
~a	0	0	1	1	0

27 bits
 0000.....011001
 32 bits
 11.....100110

0000.....0
 1111.....1 = -1

Unique Number

$$ans = 0$$

$$a \wedge 0 = a$$

$$a \wedge 0 = 0$$

Unique Number

Given a sequence of **N** non-negative numbers where every number is present **twice** except for one number which is present only **once**. Design an algorithm to find the **unique** number without using any extra space.

Example

Input : N = 7

3	1	4	1	2	3	4
---	---	---	---	---	---	---



Output : 2

=

$$3 \wedge 1 \wedge 4 \wedge 1 \wedge 2 \wedge 3 \wedge 4$$

= = = = = =

$$0 \wedge 2$$

$$= 2$$

Bitwise Operators in C++

C++ offers six different **Bitwise** operators.

Bitwise Comparator	Example	Meaning
&	a & b	Bitwise AND
	a b	Bitwise OR
^	a ^ b	Bitwise XOR
~	~a	Bitwise NOT
<<	a << b	Bitwise Left Shift
>>	a >> b	Bitwise Right Shift

Bitwise Left Shift Operator (<<)

The bitwise left shift operator **moves** the bits of its first operand to the **left** by the number of places specified by its second operand. It also inserts zero bits to fill the gaps that arise on the right side of the new bit pattern.

$$2^n \Rightarrow \boxed{1 \ll n} = 1 \cdot 2^n$$

⇒

	3rd	2nd	1st	0th	
⇒	0	0	0	1	a = 1
⇒	0	0	1	0	2
⇒	0	1	0	0	4
⇒	1	0	0	0	8

$$\begin{aligned} a &= 1 \\ a \ll 1 &= 1 \cdot 2^1 = 2 \\ a \ll 2 &= 1 \cdot 2^2 = 4 \\ a \ll 3 &= 1 \cdot 2^3 = 8 \end{aligned}$$

$$a \ll b = a * 2^b$$

Bitwise Right Shift Operator (>>)

$$\hookrightarrow \boxed{x \gg 1} \Rightarrow \frac{x}{2^1} = \frac{x}{2}$$

The bitwise right shift operator **moves** the bits of its first operand to the **right** by the number of places specified by its second operand. Moreover, it inserts zero bits to fill the gaps that arise on the left side of the new bit pattern.

$$a \gg b = \frac{a}{2^b}$$

3rd	2nd	1st	0th	
1	0	0	0	a = 8
10	0	0	0	4
100	0	0	0	2
1000	0	0	0	1

$$\begin{aligned} a \gg 1 & \quad \frac{8}{2^1} = 4 \\ a \gg 2 & \quad \frac{8}{2^2} = 2 \\ a \gg 3 & \quad \frac{8}{2^3} = 1 \end{aligned}$$

Check Odd Even

Given a non-negative integers **N**, use bitwise operators to check if it is **odd** or **even**.

Example

Input : N = 10

Output : True

Input : N = 7

Output : False

2 10
4 100
6 110
8 1000
⋮
⋮
⋮
for even nos.
0th bit is 0

3 → 11
5 → 101
7 → 111
9 → 1001
⋮
⋮
⋮
for odd nos.
0th bit is 1

101

1 (odd)
0 (even)

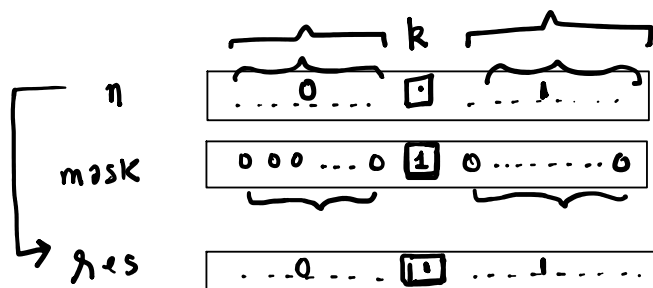
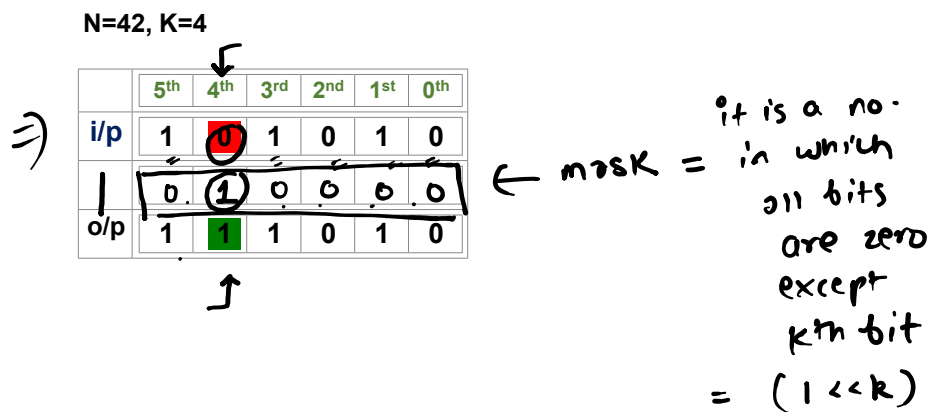
Set Kth Bit

Given a non-negative integer N , design an algorithm set its K^{th} bit.

Example

Input : $N = 42$, $K = 4$
Output : 58

$$\begin{array}{cccccc}
 5 & 4 & 3 & 2 & 1 & 0 \\
 1 & 0 & 1 & 0 & 1 & 0 & \Leftarrow 42 \\
 \hline
 1 & 1 & 1 & 0 & 1 & 0 & \Rightarrow 42 + 2^4 = 58
 \end{array}$$



$$res = n \mid mask$$

it is a no. in which all bits are zero except for the K^{th} bit

$$\begin{array}{l}
 2^0 \\
 2^1 \\
 2^2 \\
 \vdots \\
 2^K = 1 \ll K \\
 1 \cdot 2^K
 \end{array}$$

Clear Kth Bit

Clear Kth Bit

Given a non-negative integer N , design an algorithm clear its K^{th} bit.

Example

Input : $N = 42$, $K = 3$

Output : 34

5 4 3 2 1 0

1 0 1 0 1 0

$\Rightarrow 42$

1 1 0 0 1 0

$\Rightarrow 42 - 2^3$

$= 34$

$N=42, K=3$

	5 th	4 th	3 rd	2 nd	1 st	0 th
i/p	1	0	1	0	1	0
mask	1	1	0	1	1	1
o/p	1	0	0	0	1	0

$\sim =$

$\text{mask} = \sim (1 \ll K)$

this is a no-
in which all
bits are set
except for the
 K^{th} bit

Check Kth Bit

Check Kth Bit

Given a non-negative integer **N**, design an algorithm check if its **Kth** bit is set or not.

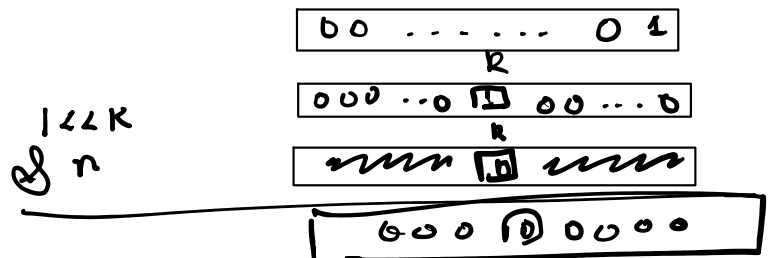
Example

⇒ Input : N = 42, K = 3
Output : true

⇒ Input : N = 42, K = 4
Output : false

5 th	4 th	3 rd	2 nd	1 st	0 th
1	0	1	0	1	0

• =



N=42, K=3

	5 th	4 th	3 rd	2 nd	1 st	0 th
	0	0	0	0	0	1
1 << K	0	0	1	0	0	0
N	1	0	1	0	1	0
&	0	0	1	0	0	0

N=42, K=4

	5 th	4 th	3 rd	2 nd	1 st	0 th
	0	0	0	0	0	1
1 << K	0	1	0	0	0	0
N	1	0	1	0	1	0
&	0	0	0	0	0	0

(1 << K) & n → 0 (Kth bit of n is not set)
→ (Kth bit of n is set)

┌┐
N=42, K=3

	5 th	4 th	3 rd	2 nd	1 st	0 th
..	.	.	.	.	.	.

N=42, K=4

	5 th	4 th	3 rd	2 nd	1 st	0 th
..	.	.	.	.	.	.

if you right-shift a no. n, k times then its Kth bit becomes 0th bit

... ..



	5th	4th	3rd	2nd	1st	0th
N	1	0	1	0	1	0
N >> K	0	0	0	1	0	1
	0	0	0	0	0	1
&	0	0	0	0	0	1

... ..

0th bit

	5th	4th	3rd	2nd	1st	0th
N	1	0	1	0	1	0
N >> K	0	0	0	0	1	0
	0	0	0	0	0	1
&	0	0	0	0	0	0

$(n \gg k) \& 1$

0 kth bit of n was not set

1 kth bit of n was set

Flip Kth Bit

Flip Kth Bit

Given a non-negative integer **N**, design an algorithm flip its **Kth** bit.

Example

Input : N = 42, K = 3

Output : 34

Input : N = 42, K = 4

Output : 58

5th	4th	3rd	2nd	1st	0th
1	0	1	0	1	0

N=42, K=3

	5th	4th	3rd	2nd	1st	0th
i/p	1	0	1	0	1	0
mask	0	0	1	0	0	0
o/p	1	0	0	0	1	0

Handwritten notes: An arrow points to the 3rd bit in the input row. The 3rd bit in the output row is boxed. Wavy lines are under the 3rd and 4th columns.

N=42, K=4

	5th	4th	3rd	2nd	1st	0th
i/p	1	0	1	0	1	0
mask	0	1	0	0	0	0
o/p	1	1	1	0	1	0

Handwritten notes: The 4th bit in the input row is boxed. The 4th bit in the output row is boxed. Checkmarks are present at the end of each row.

$$\text{mask} = 1 \ll K$$

$$\eta \wedge \text{mask}$$

mask

	.	0	1	.	0	.	0	.	0	.	0	.
o/p	✓	1	✓	1	✓	0	✓	1	✓	0	✓	✓

Count Set Bits

Given a 32-bit non-negative integer **N**, design an algorithm to count the no. of its set bits.

Example

Input : N = 42

Output : 3

5 th	4 th	3 rd	2 nd	1 st	0 th
1	0	1	0	1	0

Longest Run of 1s in Binary

Given a integer **N**, find the length of the **longest consecutive run of 1s** in its binary representation.

Example

Input : N = 247

Output : 4

	7 th	6 th	5 th	4 th	3 rd	2 nd	1 st	0 th
N	1	1	1	1	0	1	1	1

Handwritten annotations: A bracket above the 2nd, 1st, and 0th bits is labeled '3'. A bracket below the 7th, 6th, 5th, and 4th bits is labeled '4'.

Hamming Distance



Given two 32-bit integers **N** and **M**, design an algorithm to find the **Hamming Distance** between them. The Hamming distance between two integers is the number of positions at which the corresponding bits are different.

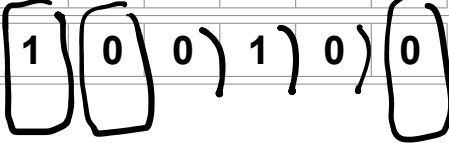
Example

Input : N = 42 M = 36

Output : 3



	5 th	4 th	3 rd	2 nd	1 st	0 th
N	1	0	1	0	1	0
M	1	0	0	1	0	0



Power of 2

Given a non-negative integer **N**, design an algorithm to check if it is a **power of 2**.

Example

Input : N = 8
Output : True



$$8 = 2^3$$

Input : N = 10
Output : False



$$16 = 2^4$$

Power of 4

Given a 32-bit non-negative integer **N**, design an algorithm to check if it is a **power of 4**.

Example

Input : N = 16

Output : True

Input : N = 8

Output : False

Compound Assignment Operators

$$a = a + b \Rightarrow a += b;$$

Compound Assignment Operators in C++

Operator	Example	Equivalent to
+=	<code>a += b</code>	<code>a = a + b</code>
-=	<code>a -= b</code>	<code>a = a - b</code>
*=	<code>a *= b</code>	<code>a = a * b</code>
/=	<code>a /= b</code>	<code>a = a / b</code>
%=	<code>a %= b</code>	<code>a = a % b</code>
&=	<code>a &= b</code>	<code>a = a & b</code>
=	<code>a = b</code>	<code>a = a b</code>
^=	<code>a ^= b</code>	<code>a = a ^ b</code>
<<=	<code>a <<= b</code>	<code>a = a << b</code>
>>=	<code>a >>= b</code>	<code>a = a >> b</code>

$$a = a \quad \underline{\text{op}} \quad b$$

$$\left\{ \begin{array}{l} +, -, *, /, \% \\ \&, |, ^, \gg, \ll \end{array} \right.$$

$$a \quad \text{op} = \quad b ;$$